CS-1101
PROGRAMMING FUNDAMENTALS

Lab 04

By

Engr. Muhammad Awais

November 24, 2024

Lab Contents

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Ternary or Conditional Operators

Lab Objectives

- Apply arithmetic, relational, logical, assignment and ternary operators in solving real-world problems,.
- Apply operator precedence when combining different types of operators in expressions, ensuring accurate evaluation and output.

Arithmetic Operators

- These operators are used to perform mathematical operations on the operands.

Name	Symbol	Description	Example
Addition	+	Adds two operands	int a = 3, b = 6; int c = a+b; // c = 9
Subtraction	–	Subtracts second operand from the first	int a = 9, b = 6; int c = a-b; // c = 3
Multiplication	*	Multiplies two operands	int a = 3, b = 6; int c = a*b; // c = 18
Division	/	Divides first operand by the second operand	int a = 12, b = 6; int c = a/b; // c = 2
Modulo Operation	%	Returns the remainder an integer division	int a = 8, b = 6; int c = a%b; // c = 2

The Modulo operator(%) operator should only be used with integers.

Relational Operators

- These operators are used for the comparison of the values of two operands. The result returns a Boolean value, i.e., true or false.

Name	Symbol	Description	Example

Is Equal To	==	Checks if both operands are equal	int a = 3, b = 6;
			a==b;
			// returns false

Greater Than	>	Checks if first operand is greater than the second operand	int a = 3, b = 6;
			a>b;
			// returns false

Greater Than or Equal To	>=	Checks if first operand is greater than or equal to the second operand	int a = 3, b = 6;
			a>=b;
			// returns false

Less Than	<	Checks if first operand is lesser than the second operand	int a = 3, b = 6;
			a<b;
			// returns true

Less Than or Equal To	<=	Checks if first operand is lesser than or equal to the second operand	int a = 3, b = 6;
			a<=b;
			// returns true

Not Equal To	!=	Checks if both operands are not equal	int a = 3, b = 6;
			a!=b;
			// returns true

Logical Operators

- These operators are used to combine two or more conditions or constraints. The result returns a Boolean value, i.e., true or false.

Name	Symbol	Description	Example

Logical AND	&&	Returns true only if all the operands are true or non-zero	int a = 3, b = 6;
			a&&b;
			// returns true

Logical OR	||	Returns true if either of the operands is true or non-zero	int a = 3, b = 6;
			a||b;
			// returns true

Logical NOT	!	Returns true if the operand is false or zero	int a = 3;
			!a;
			// returns false

Assignment Operators

- These operators are used to assign value to a variable. The left side operand of the assignment operator is a variable, and the right-side operand of the assignment operator is a value.

Name	Symbol	Description	Example
Assignment Operator	=	Assigns the value on the right to the variable on the left	int a = 2; // a = 2
Add and Assignment Operator	+=	First adds the current value of the variable on left to the value on the right and then assigns the result to the variable on the left	int a = 2, b = 4; a+=b; // a = 6
Subtract and Assignment Operator	-=	First subtracts the value on the right from the current value of the variable on left and then assign the result to the variable on the left	int a = 2, b = 4; a-=b; // a = -2
Multiply and Assignment Operator	*=	First multiplies the current value of the variable on left to the value on the right and then assign the result to the variable on the left	int a = 2, b = 4; a*=b; // a = 8
Divide and Assignment Operator	/=	First divides the current value of the variable on left by the value on the right and then assign the result to the variable on the left	int a = 4, b = 2; a /=b; // a = 2

Ternary or Conditional Operators(?:)

- This operator returns the value based on the condition.
- This operator takes three operands; therefore, it is known as a Ternary Operator.

condition ? expression 1 : expression 2

Operator Precedence

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

Evaluate following expressions

#	Expression
1	5 + 3 * 2
2	8 / 4 + 6 * 3
3	(10 + 2) * 5 - 6 / 2
4	9 - 3 + 2 * 4
5	15 / (3 + 2) * (6 - 4)
6	10 + 6 / 2 * 3 - 4
7	7 + (3 * 4) - (6 / 2)
8	5 + 10 / 4 * 3 -2
9	5 + 10.0 / 4 * 3 - 2
10	(25 / (3 + 2)) * 4 - (18 % 5) + 6
11	(10 + 5 > 3 * 4)
12	5 * 3 > 20 || 12 % 5 == 2
13	x = 5, y = 3; x++ + ++y > 10
14	m = 2, n = 3, o = 4 m++ + ++n * ++o

Task 1 : Determine Even or Odd Numbers

- Write a program that checks if an integer inputted by user is even or odd using the ternary operator.

- **Format the output to display:**
  - "The number X is even" if the number is even. "The number X is odd" if the number is odd.

Input Example:

- Enter an integer : 5
- Enter an integer : 12

Output Format Example:

- The number 5 is odd
- The number 12 is even

Task 2 : Age Group Classification

- Write a program that classifies a person's age into different categories:
  - "Child" if the age is less than 13.
  - "Teenager" if the age is between 13 and 19.
  - "Adult" if the age is 20 or greater.

  Use the ternary operator to classify the age and display the output.

  Input Example:
  - 10
  - 15
  - 25

  Output Format Example:
  - Age 10 falls under the category: Child
  - Age 15 falls under the category: Teenager
  - Age 25 falls under the category: Adult

Task 3 : Banking App

- Write a program that checks a customer's bank account balance and determines both their account type and special offer eligibility.
- The program should work as follows:
  - **Account Type Determination:**
    - If the account balance is less than $100, the account is considered a Low Balance account.
    - If the account balance is between $100 and $500 (inclusive), it is considered a Standard Account.
    - If the account balance is greater than $500, it is considered a Premium Account.
  - **Special Offer Eligibility:**
    - The customer is eligible for a special offer if their account balance is greater than $200 and they are also a loyal customer. The loyalty status is provided as a Boolean value (1 for loyal, 0 for not loyal).
  - **Output Format:**
    - Print the Account Type based on the account balance.
    - Print whether the customer is eligible for a special offer based on their balance and loyalty status.

You are required to use the ternary operator to implement all the decision-making logic in your program. You should not use if-else statements. Additionally, use logical operators (AND, OR, NOT) where needed to combine multiple conditions.

REFERENCES

- https://www.geeksforgeeks.org/operators-in-cpp/